Our problem and approach

Track B final project report

Vanshika Tiwari | Harshid Bhinde | Hritwik Ekade
94-879 Fundamentals of Operationalizing AI | Track B | October 10, 2026

We recommend using B2's corrected model path and readiness check. B1's append fix is faster, but we would only use it for tests where the input can be replayed until its crash-recovery problem is fixed. For B3, we recommend investigating the drift before deciding whether to retrain. Each challenge shows a different way the system can look healthy while something is wrong.

The system and the evidence we kept

The system sends recorded market ticks through Kafka, calculates rolling features, saves them in Parquet, and serves model predictions through an API. We covered B1, B2 and B3 in our presentation. After the feedback, we added the tests in this report. We still have the September 25 B2 logs, but not the original B1 logs or B3 model and script. We use new measurements where we could not verify the old ones.

B1: what we thought was causing the lag

We first thought adding partitions and consumers would reduce lag. We said we would reject this idea if lag kept growing at the same rate. Our presentation reported that result. The producer uses the same BTC-USD key for every message, so all messages go to one partition and the extra consumers can sit idle. We did not repeat that partition experiment for this report, and its original logs are unavailable.

Our next hypothesis was that rewriting the full Parquet file every 25 feature rows was slowing the consumer down. If save time did not increase with the rows already on disk, or append did not help under the same load, we would question this explanation. The code repeatedly serializes all earlier rows, giving quadratic total work as the file grows. A drop in throughput during one run was not enough to isolate that cause.

Testing save time at different file sizes

We ran seven trials for each writer and file size, in random order. Each trial added the same 25 rows using the same schema, computer and storage. We prepared the existing file before starting the timer, so we could vary file size without relying on how long the run had been going. The append timing does not include closing its footer, which is an important difference in durability.

Rows already stored	Rewrite median (ms)	Append median (ms)
0	3.94	3.72
5,000	22.68	1.04
20,000	26.93	1.01
80,000	46.44	1.15

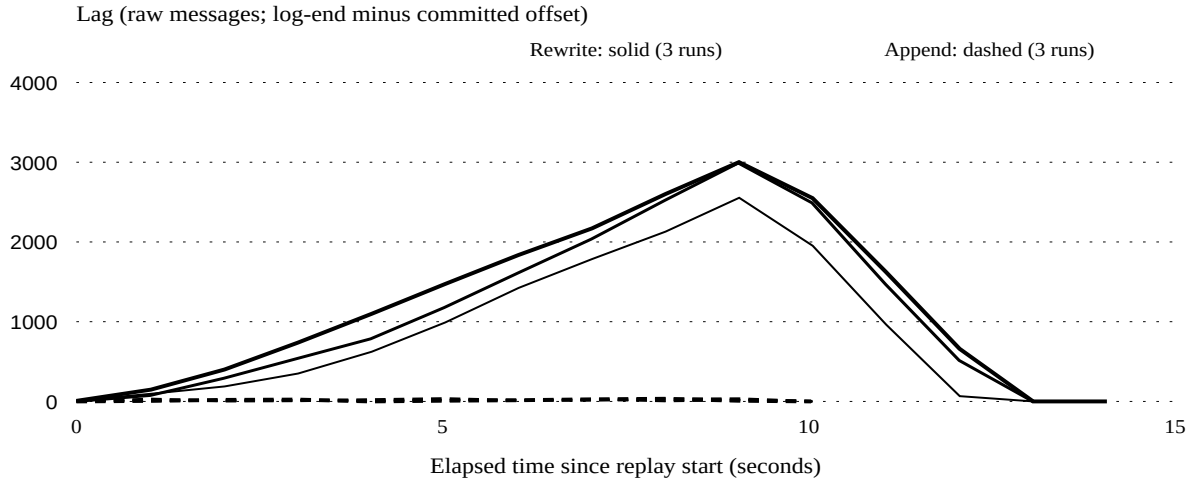
The rewrite took longer as the stored row count increased. After initialization, append time stayed close to 1 ms. This supports our explanation of the code, although the timings alone do not prove an asymptotic growth rate. These tests used a warm cache and did not test fsync or power loss.

Evidence: b1_save_trials.csv (56 trials); b1_save_summary.csv; supplied baseline _append_parquet and submitted _ParquetAppender.

B1: results and the crash problem

Repeating the same workload

For each run, we used a new empty topic and consumer group, one partition, one consumer and a fresh output file. We replayed the same 14,076 messages once, without looping. They produced 11,754 feature rows because non-ticker messages were skipped. Timestamps never moved backward. We tested each writer three times, alternated the order, and ran the final timings without other benchmarks or bootstrap calculations running.



Measure (3 runs each)	Rewrite	Append
Achieved raw arrivals/s	1,499.8-1,499.9	1,499.7-1,499.8
Peak sampled lag (messages)	2,554-3,001	23-35
Observed completion (seconds)	13.09-14.11	10.12-10.13
Features / observed second	833.1-898.1	1,160.4-1,161.4
Final sampled lag	0 in all runs	0 in all runs
Exact feature-row equality	11,754 / 11,754	11,754 / 11,754

We calculated the arrival rate from acknowledged messages divided by producer time, including the final flush. We sampled lag about once a second, so completion times include some observation delay. Every output row and column matched the direct single-pass calculation exactly. This shows an improvement on this finite workload; it does not tell us the capacity over several hours.

What happened when we used kill -9

We sent Linux SIGKILL after 40 completed batches of 25 rows. The rewrite file still had 1,000 readable rows, but the append file had zero rows readable by PyArrow because its footer was missing. This affected the whole open file, not just the last row group. This test only checked the writer and did not commit Kafka offsets. The actual consumer commits after appending, so it could commit offsets for output that becomes unreadable after a crash.

We would keep the SIGTERM/SIGINT handler because it closes the writer during a normal shutdown. It cannot help with SIGKILL. The old rewrite method can also corrupt a file if it is killed during a write. For now, we recommend append only when the input is retained and can be replayed, with a new output path each time. Durable file segments and recovery of data and offsets still need to be tested.

Evidence: b1_stream_summary.csv; b1_stream_telemetry.csv and six logs; b1_sigkill.json. Original 73,607 lag and 3,000/s achieved-rate claims are withdrawn: looping or pre-existing backlog cannot be distinguished without the missing logs.

B2: fixing the model path and health check

Finding the cause

We found that MODEL_PATH used /app/models/artifact/model.pkl, but the mounted folder was named artifacts. The original scorer caught the missing-file error and kept running. Docker checked /health, which returned 200 even when /score returned 503. The wrong path caused the scoring failure, and the health check failed to detect it.

Our hypothesis was that correcting only MODEL_PATH would restore scoring with the same image, volume and request payload. If scoring still returned 503, the path would not explain the failure. The saved logs show that the configured path did not exist, the mounted model did exist, and changing only the path restored HTTP 200 and the baseline score. Restarting three times with the wrong path did not fix it.

What we changed

We corrected the model path and kept /health as a simple check that the process is responding. We added /ready to check that the model is loaded and run a canary through the same predict function as /score. It only returns 200 if the prediction is finite and between 0 and 1. Otherwise, it returns 503 and explains the failure. Docker now checks /ready, so a model-loading failure no longer looks healthy.

Controlled condition	Docker	/health	/ready	/score
Original + bad path	healthy	200	absent	503
Repaired + hardened	healthy	200	200	200
Hardened + bad path	unhealthy	200	503	503

In the saved test, we checked Docker's status every 10 seconds. It showed starting from 0 to 40 seconds, then unhealthy at 50 seconds after five failed probes. /health stayed at 200 after startup. We also tested a corrupt model: the original scorer exited with code 1, while the updated scorer stayed up and reported UnpicklingError through /ready.

Checking the result and probe cost

The earlier latency test used 300 sequential requests per endpoint after 20 warm-ups. /health had p50 3.50 ms and p95 5.51 ms; /ready had p50 4.65 ms and p95 6.03 ms. The median difference was 1.15 ms. These were local sequential requests, so they do not show the cost under concurrent load. We have the summary, but not the individual timings.

We added the locally tested scorer and corrected Compose files to the final repository. Using Flask's test client and the new model, we checked /health, /ready and /score again. They returned 200/200/200 with a valid model and 200/503/503 with missing or corrupt models. These new checks test the application; they are not a repeat of the Docker timing experiment.

What we recommend and what is still untested

We recommend both changes. In Compose, unhealthy is just a status label; it does not stop traffic or restart the container. A canary with outdated inputs could mark every replica unready, and a working but incorrect model could still pass. We would monitor ready replicas, scoring errors, model-load errors and startup time. Traffic removal and failed rollouts would need separate Kubernetes tests.

Evidence: evidence/b2_historical/01-08 captures, especially 03_diagnosis, 05_fault_injected_hardened_code and 07_probe_cost; supplemental/results.json, b2 section.

B3: checking drift and model performance

Why we partly rejected our hypothesis

We expected wider spreads to cause drift in spread_normalized and affect the model. Our original rejection rule included the case where drift appeared but the model was unaffected. We detected drift, but we did not show clear model degradation, so our hypothesis was partly rejected. We still recommend investigating because the inputs changed substantially and this small synthetic test cannot show that the change is harmless.

Was the spread change 4x or 5x?

We used the same featurizer on both files. Each had 782 raw messages and 653 feature rows. Mean normalized spread increased 3.9792x, and mean raw spread increased 3.9792x. The first tick changed from 0.01 to 0.05, which is 5x, but the ratios across ticks ranged from about 3x to 5x. This explains the slide discrepancy: the average change was about 4x, while the example was 5x.

Feature	KS statistic	Interpretation
spread_normalized	0.9525	Large distribution shift
midprice_returns	0.0046	Small rounding perturbation
order_book_imbalance	0.0000	Unchanged
trade_intensity	0.0000	Unchanged

We used SciPy to check the KS results from the presentation. The spread p-value was too small to represent numerically; the other p-values were 1. Since the rows are time-dependent, we put more weight on the size of the change than the p-value. Only one of four features changed substantially, so the 50% dataset-level rule could miss the spread change.

Testing a new model on the two inputs

We did not have the original model from the slides, so we fitted a new balanced logistic regression and scaler on the first 371 rows. We left a 20-row gap before the 60% training boundary. The test set used rows 522-632, giving 111 paired rows after removing the last 20 rows without future labels. We labeled spikes using forward 20-row return volatility and the reference-wide 90th percentile. We kept these labels fixed for both inputs; 19 test rows were positive.

These labels use the full reference window, so this is a retrospective comparison. A cutoff based only on training data gave no positive test cases. Trade prices were unchanged, but rounding the bid and ask slightly changed midprice returns (maximum absolute difference 9.85e-8). We did not use test rows to fit the model or scaler.

Average precision (AP)	Estimate	95% block-bootstrap interval
Reference	0.208	[0.031, 0.502]
Current	0.254	[0.037, 0.576]
Current minus reference	0.046	[0.000, 0.163]

We used sklearn's average_precision_score for AP, which we call PR-AUC here; it is not the trapezoidal area under the PR curve. We tried 3,000 paired circular block-bootstrap samples with block length 20. Of these, 2,732 had both classes. The interval for the difference includes zero. These intervals apply to the new model, not the slides' 0.208/0.303 result, so we cannot label that earlier result as noise.

Evidence: results.json; b3_drift.csv; b3_paired_scores.csv; b3_bootstrap.csv. Block length 10 sensitivity: delta 95% interval [0.000, 0.171], 2,902 valid draws. Intervals omit model-training uncertainty.

What we would monitor and do next

Trying different alert thresholds

Threshold	Alerts: ref / current	Precision: ref / current	Recall: ref / current
0.3	108/111 / 107/111	0.157 / 0.150	0.895 / 0.842
0.5	100/111 / 98/111	0.170 / 0.153	0.895 / 0.789
0.7	72/111 / 70/111	0.181 / 0.186	0.684 / 0.684
0.9	0/111 / 0/111	undefined / undefined	0.000 / 0.000

At 0.5, 90.1% of reference rows and 88.3% of current rows were flagged, so alert rate alone tells us little. Recall fell from 0.895 to 0.789 despite higher AP; saying the model was unaffected was too strong. At 0.7, alert rates were 64.9% and 63.1%, with recall 0.684 in both. At 0.9 there were no alerts. These comparisons explore sensitivity; we did not use them to choose a threshold.

The mean absolute score change was 0.0153. Results depend on the threshold, and the intervals are wide. We cannot conclude that production performance would be better or worse. We would keep serving while investigating, rather than retrain from a drift p-value or one AP estimate.

Our suggested monitoring plan

Signal	Trigger / response	Owner
Infrastructure / readiness	Zero ready replicas or scoring 5xx: page; inspect path/artifact, restore known-good config.	Service operator
B1 lag + freshness	Sustained lag growth or stale output: inspect save cost; reduce intake or replay safely.	Pipeline owner
Feature drift	Proposed: KS effect >0.2 AND spread mean ratio >1.5; investigate feed/units and market context.	Model + data owner
Model quality	Collect delayed labels; compare precision/recall and AP with uncertainty and operating cost.	Model owner

This drift threshold is a starting point; we have not measured its false-alarm rate. We would set a minimum window size, require persistent changes and avoid duplicate alerts. We would track health, drift and model quality separately, since the dataset rule can miss a large change in one feature.

Before using these changes in production

B1: keep the source data and use a fresh output path for each run. The current writer can overwrite an existing file after a restart. Before production, we need closed, durable file segments, a clear plan for recovering data and offsets, and tests that kill the process during a write and then restart it. A committed Kafka offset does not by itself prove the output is safely stored.

B2: use the path repair and readiness check after checking that the canary inputs match the model. Keep a rollback procedure because restarting cannot fix a missing model. Check model identity separately: /ready shows that prediction runs, not that it is the intended model. We would also pin tested image digests before production because the starter uses Kafka's latest tag.

B3: check the data source and units, save both windows, and collect labels from a real period with wider spreads. We would retrain only after finding a sustained, meaningful drop in labeled performance and testing a replacement. Too many drift alerts could waste operators' time, but ignoring a large change in one feature could hide a data problem.

Evidence: b3_thresholds.csv and b3_paired_scores.csv. Proposed operational triggers are recommendations, not tested SLAs or production guarantees.

How we used AI and what is still open

AI example 1: rerunning B1 with better controls

AI suggested replacing the B1 slide numbers we could not verify with new controlled experiments. We accepted this during final preparation. The reason was that a clean, one-pass workload of 14,076 messages could not explain lag of 73,607. We also could not confirm that the configured rate of 3,000/s was achieved without the original logs.

We checked this recommendation by running six tests with new Kafka topics and groups. Each acknowledged all 14,076 messages, recorded the achieved rate, and had no backward timestamps. All 11,754 feature rows matched the direct calculation exactly. We also timed saves at fixed file sizes. These tests gave us evidence for the final report, but they do not recover or validate the missing original runs.

AI example 2: changing our B1 recommendation

AI also suggested changing our recommendation from 'ship append with a shutdown handler' to using it only where replay is possible until durability is fixed. We accepted this after reviewing the SIGKILL result. A normal shutdown handler cannot run after SIGKILL, and without the footer the open Parquet file cannot be read normally.

We checked this with a test that completed 40 batches (1,000 rows) and then sent Linux SIGKILL. The rewrite file had 1,000 readable rows; the append file had none. We changed our recommendation and removed the claim that the old method was always crash-safe. This test checked file readability, not recovery of committed Kafka data. We are responsible for explaining that difference and for the final recommendation.

How to repeat our tests

To repeat the tests, install experiments/requirements.txt with Python 3.12 and run experiments/validate_submission.py from the repository root. Start the Kafka broker and run experiments/stream_comparison.py --rate 1500 for the streaming comparison. On Linux, run validate_submission.py --crash for the kill test. The README has the Docker setup, B2 fault test and expected outputs.

We included the scripts, input hashes, trial results, lag logs, paired scores and bootstrap samples. The older B2 logs are dated September 25, and the new tests are dated October 10. The original code is available in the challenge package and initial commit. The evidence files contain the details behind the tables in this report.

Our main unanswered question

Our biggest remaining question is whether the faster B1 pipeline can recover all committed output after a crash and restart. The speed improvement is useful, but we cannot call it production-ready until we test a reliable way to save data and commit offsets. For B3, we still need labels from a real wide-spread period. The synthetic input change and retrospective labels cannot tell us how useful the model would be there.

Sources and supporting files

We used the assignment, 94-879 Final Team Project Track B - Technical Challenges, and the professor's feedback to plan the final tests. This report updates the results in our presentation, When Healthy Isn't Working. Our repository is vanshika1631/OpsAI_TrackB (initial commit eae698f). Details about the evidence are in evidence/README.md, and our response to each feedback item is in submission/feedback_response.md.

These examples come from the final-preparation work. Both recommendations were accepted in that discussion.